Binary search trees

Part III Data Structures

Scope

A BST turns global key order into a root-to-leaf query path, but an ordinary BST does not balance itself. The same ten-key tree aligns search, successor, and every parent update in two-child deletion. A direct height-moment proof for random insertion distinguishes complete traversal, average depth, and height.

Prerequisites

Recursion, linked nodes, and random permutations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Build and query binary search trees.
- Implement insertion and all three deletion cases.
- Analyze the expected height of a randomly built tree.
- 12.1 What is a binary search tree?
- 12.2 Querying a binary search tree
- 12.3 Insertion and deletion
- 12.4 Randomly built binary search trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The global search-tree invariant

Each node stores key, left, right, and parent. The main algorithms use distinct keys: every key in x's left subtree is less than x.key, and every key in its right subtree is greater. The root has parent=NIL, missing children are NIL, and an empty tree has root=NIL. The structure must be connected and acyclic, with exactly one parent per nonroot node.

The condition constrains entire subtrees, not just direct children. A root 5 with left child 3 and 3's right child 6 satisfies 3<5 and 6>3, but 6 violates the root's left-subtree bound. Validate by recursively passing an allowed interval (lo,hi), setting hi=x.key on the left and lo=x.key on the right. Checking every node takes Θ(n), without rescanning subtrees at each node.

Duplicates can use a count per distinct key or a total order on (key,identity). With the latter, adjacent identities can have the same key, so successor no longer means a strictly larger key. All numerical cases here use distinct keys, as does the random-height analysis.

Sorted inorder and complete traversal cost

```
inorder(x):
    if x==NIL: return
    inorder(x.left)
    output x.key
    inorder(x.right)
```

Inductively, each subtree's inorder is sorted, and every left key<x.key<every right key. Concatenating left, root, right is sorted. Conversely, a valid binary tree with distinct keys and strictly increasing inorder satisfies the global BST condition, since all left keys precede the root and all right keys follow it.

Each of n real nodes calls both children once, giving exactly 2n+1 calls including NIL. Constant work per call gives Θ(n+1), conventionally Θ(n) for a nonempty tree, not O(h). Producing n outputs also gives Ω(n). Recursion depth is h+1 and can be Θ(n) on a chain. We use edge height throughout: a singleton has h=0 and an empty tree h=−1. Nonempty path operations take O(h+1); empty-tree search and first insertion take O(1).

Sorted inorder and complete traversal cost (continued)

A heap orders parents against children but does not place all left keys below the root and all right keys above it. Its inorder need not be sorted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

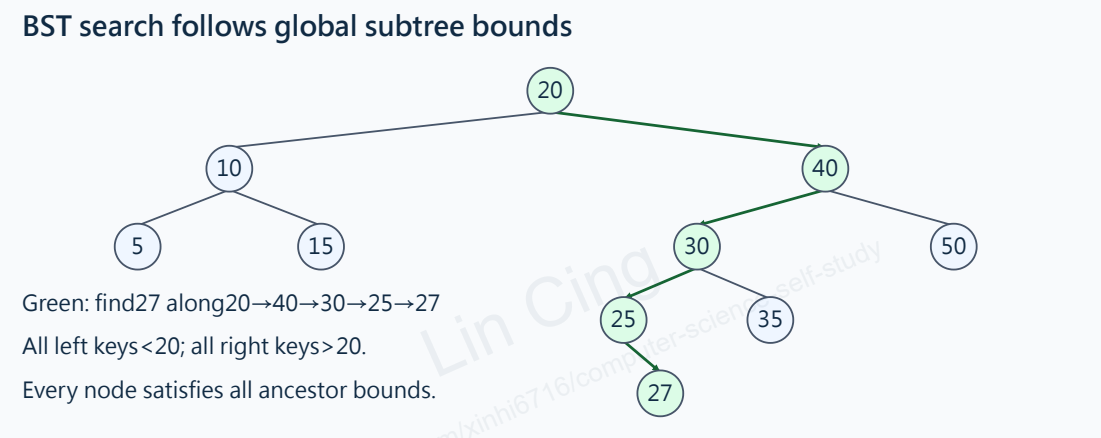
Querying a binary search tree

Search compares at the root: return on equality, go left for a smaller query and right for a larger one, and reject at NIL. Its loop invariant is that any existing answer lies in the current subtree. Global BST bounds justify discarding the other side. Each step descends one level, giving O(h+1) time and O(1) iterative extra space.

Minimum follows left links in a nonempty subtree until none remains. A left child supplies smaller keys; without one, the root is no greater than anything in its right subtree. Maximum reverses left and right.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Querying a binary search tree (continued)



search27 follows 20→40→30→25→27. search26 reaches 27, goes left to NIL, and fails. Minimum is 5 and maximum is 50. A query follows a path; complete inorder is 5,10,15,20,25,27,30,35,40,50. Their costs differ.

The two successor branches

If x.right is nonempty, successor is min(x.right): all right-subtree keys exceed x, and its minimum precedes every larger candidate there. Otherwise climb parents, skipping edges where the current node is a right child. The first parent entered from its left child is the answer; NIL means none exists.

```
successor(x):
    if x.right!=NIL: return minimum(x.right)
    y=x.parent
    while y!=NIL and x==y.right:
        x=y; y=y.parent
    return y
```

Skipped parents are smaller than the original x and cannot be successors. At the first left ascent, y has original x in its left subtree and is larger. No unexcluded key after x remains in y's left subtree: x had no right child, and the intervening ascents followed final right branches. Thus y is the smallest larger key. Predecessor exchanges left and right.

The two successor branches (continued)

successor20 follows the left chain 40→30→25 and returns 25. successor27 skips 25 on a right ascent, then enters 30 from the left and returns 30. successor50 skips 40 and 20 and reaches NIL. No new root search or sorting is needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Successive successors share traversal work

Starting at minimum, repeated successor lists all n keys in $\Theta(n)$ total time. Over the complete inorder, each child edge is traversed downward at most once and upward at most once. Add n outputs for $O(n)$. One $\Theta(h)$ successor does not imply paying that path again at every node.

For k successive calls from any starting node, stopping at NIL, count upward steps U and downward steps D. Each call has at most one ascent from a left child to its parent, which is that call's answer, for at most k such edges. Every other ascent is from a right child. Its parent was output before entering its right subtree; after climbing above it, traversal never returns to that subtree, so each such parent is charged once.

Previously output parents from before these k answers must lie on the starting node's ancestor chain, at most h+1 nodes; other charged parents number at most k. Hence $U \leq 2k+h+O(1)$. Downward steps increase depth and upward steps decrease it. $D-U$ is final depth minus initial depth, at most $h+O(1)$, giving $D \leq U+h+O(1)$. Total path work plus k fixed call costs is $O(k+h+1)$, without assuming constant work per ascent or descent.

Successive successors share traversal work (continued)

This refines, but does not change, the single-call O(h+1) bound and explains why complete traversal is not O(nh). Reimplementing each successor as a new root search is a different algorithm and cannot use this shared-edge accounting.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Insertion

Follow the search path to its final NIL, retaining parent p. Initialize z.left=z.right=NIL and z.parent=p, then attach z to p.left or p.right according to comparison. If p=NIL, the tree was empty and root=z.

The path's accumulated interval (lo,hi) is exactly the allowed range for z.key. Reaching NIL means no equal key exists. Adding the leaf leaves other keys unchanged and satisfies every ancestor bound, preserving the BST invariant. On equality, update the existing dictionary record rather than adding a duplicate.

Inserting 26 into our tree follows 20→40→30→25→27 and reaches 27.left=NIL. Set 27.left=z26 and z26.parent=27. It must satisfy 25<26<27. Merely knowing 26>25 does not justify overwriting 25's right subtree rooted at 27.

Insertion takes worst-case O(h+1). Inserting 1,2,…,n into an empty tree creates a right chain with $0+1+\ldots+(n-1)=\Theta(n^2)$ comparisons. A later O(n) inorder does not make this a O(n log n) sorting algorithm.

Transplant replaces an incoming link

```
transplant(T,u,v):
    p=u.parent
    if p==NIL: T.root=v
    elif u==p.left: p.left=v
    else: p.right=v
    if v!=NIL: v.parent=p
```

Transplant gives v u's former position under its parent in O(1). It does not set v.left/v.right or dispose of u's satellite data. The caller must prove that v's subtree fits the incoming link's global key interval; an arbitrary subtree cannot be transplanted safely.

If z has no left child, delete with transplant(z,z.right), allowing right=NIL and covering leaves and right-only nodes. If z has no right child, use transplant(z,z.left). The surviving subtree already satisfies z's ancestor bounds, so promotion preserves order and updates parent links. The same helper handles root deletion.

Complete pointer specification for two-child deletion

```
delete(T,z):
    if z.left==NIL: transplant(T,z,z.right)
    elif z.right==NIL: transplant(T,z,z.left)
    else:
        y=minimum(z.right)
        if y.parent!=z:
            transplant(T,y,y.right)
            y.right=z.right
            y.right.parent=y
        transplant(T,z,y)
        y.left=z.left
        y.left.parent=y
```

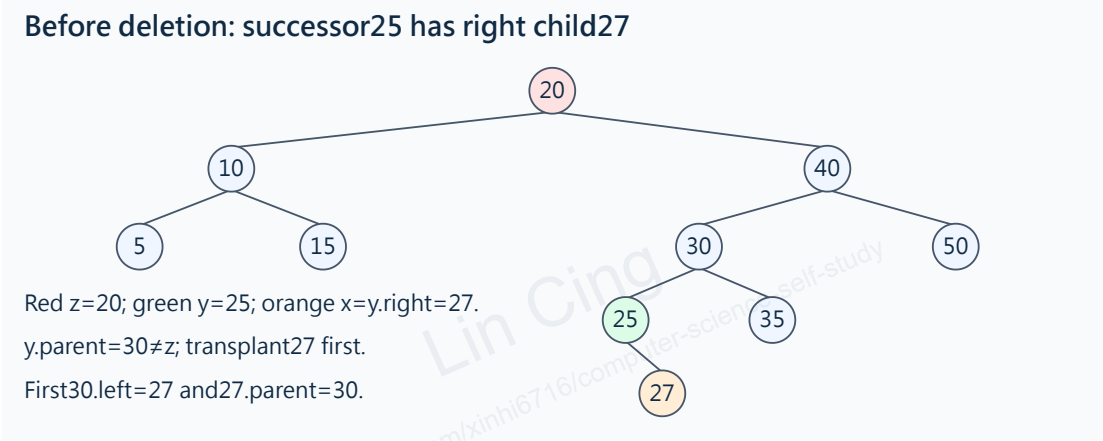
As the right-subtree minimum, y has no left child, but may have a right child that must survive. If y is not z.right, promote y.right to its old position, then give y the original right subtree. If y is z.right, skip that branch: y.right=z.right would create a self-cycle, and y's existing right child must remain intact.

Complete pointer specification for two-child deletion (continued)

Finally y replaces z under its old parent, or becomes root, and receives the original left subtree. Every new left key<z<y. The new right subtree is the old right subtree minus y, with all keys>y. Since y belonged to z's subtree, it also satisfies all of z's ancestor bounds. Thus global BST order, reachable node identities, and parent directions are correct.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Preserve a successor's right child



Here z=20 and y=25 is 30.left, not direct right child40. y.right=27. Apply these updates in order.

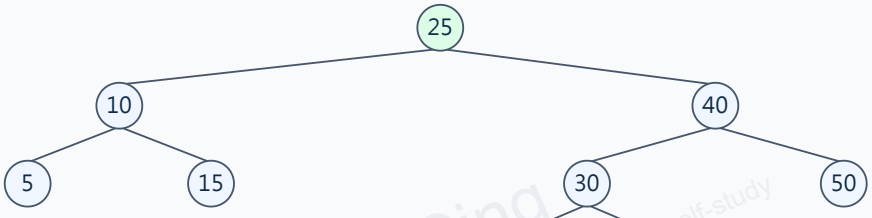
Preserve a successor's right child (continued)

Stage	Links actually changed
Replace y's old position	30.left=27, 27.parent=30
Attach original right subtree	25.right=40, 40.parent=25
Replace original root	root=25, 25.parent=NIL
Attach original left subtree	25.left=10, 10.parent=25

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Preserve a successor's right child (continued)

After deletion: reconnect both positions



Green25 becomes root; parent=NIL.

Orange27 becomes30.left; parent=30.

Only20 is removed; all nine other identities remain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Only node20 is removed. All other nine identities remain reachable, with inorder 5,10,15,25,27,30,35,40,50. This moves node25; it does not mix in the different handle semantics of copying a key and deleting another node.

Handles and deletion cost

z must be a currently valid node in this tree. After deletion, external callers may not treat z as a member. Other node identities remain, including y25, whose handle still denotes the same record at a new position. Copying y's key/value into z and deleting y gives external handles different observations and needs a separate API specification.

Deletion has one O(h+1) minimum path and only a fixed number of transplants and pointer writes, so takes O(h+1). Starting from a key also requires search, of the same order, not O(1). Search, insertion, and deletion depend on height; ordinary BSTs have no automatic balancing guarantee.

A fixed sorted array can be converted into a balanced tree by recursively choosing midpoints in Θ(n). For arbitrary unsorted comparison inputs, constructing a complete BST and reading inorder would solve sorting, giving a general worst-case construction lower bound Ω(n log n). These are different input contracts.

Randomly built binary search trees

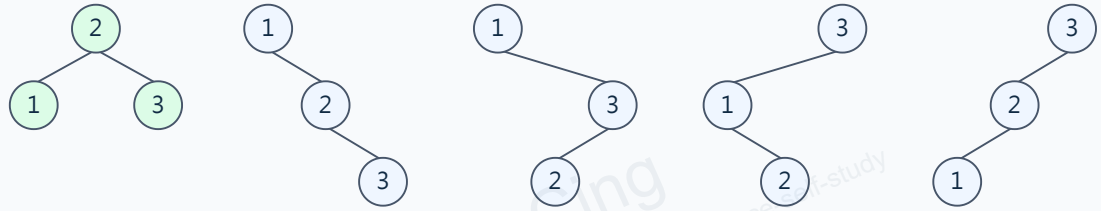
Fix n distinct keys, start empty, choose each of n! insertion permutations equally, and perform insertions only. This differs from assigning equal probability to every valid BST shape. The first key becomes root, so root rank R is uniform on 1..n.

Conditioned on R=r, the left side contains r−1 smaller keys and the right n−r larger keys. Each specified pair of internal relative orders has binom(n−1,r−1) interleavings. Thus, conditional on the root, both relative insertion orders are uniform and mutually independent.

Randomly built binary search trees (continued)

Uniform permutations weight tree shapes

Six permutations are uniform; five shapes are not. Singleton edge height0.



Green height1; others height2. E[H3]=(2×1+4×2)/6=5/3.

For n=3, [2,1,3] and [2,3,1] yield the balanced shape rooted at2, with probability 2/6. Each of four chain shapes has probability 1/6. Heights are 1 or 2, so $E[H_3]=(2\times1+4\times2)/6=5/3$, not 9/5 from uniform shapes.

Control exponential height directly

O(log n) expected depth of one node does not directly imply O(log n) expected maximum depth. Instead bound an exponential of height.

Let H_n be edge height, $H_0=-1$, and Z_n=2^(H_n). Then $Z_0=1/2$ and $Z_1=1$. Unlike the textbook's convenient separate convention $Z_0=0$, we keep Z=2^H valid even for the empty tree and use correspondingly looser constants.

Conditioned on left-subtree size i, H_n=1+max(H_i,H_(n−1−i)), hence `Z_n=2 max(Z_i,Z_(n−1−i))≤2(Z_i+Z_(n−1−i))`.

Write a_n=E[Z_n]. Conditional subtrees are uniform for their sizes, so averaging over i gives

`a_n ≤ (2/n)Σ_(i=0)^(n−1)(a_i+a_(n−1−i)) = (4/n)Σ_(i=0)^(n−1)a_i.`

This uses max≤sum followed by linearity, not the false equality E[max(X,Y)]=max(E[X],E[Y]).

A polynomial moment bound gives logarithmic height

Set b_n=(1/2)binom(n+3,3), with $b_0=1/2=a_0$. The hockey-stick identity Σ_(i=0)^(n−1)binom(i+3,3)=binom(n+3,4) yields

$$(4/n)\Sigma_{i=0}^{n-1} b_i = (2/n)\binom{n+3}{4} = (1/2)\binom{n+3}{3}=b_n.$$

The identity telescopes Pascal's relation binom(i+4,4)−binom(i+3,3)=binom(i+3,3). Induction from a_i≤b_i gives $a_n \le b_n = O(n^3)$, including $a_1=1\le b_1=2$.

Since $\log_2$ is concave, Jensen gives $E[H_n]=E[\log_2 Z_n]\le \log_2 E[Z_n]\le \log_2 b_n = O(\log n)$. Conversely, a height-h binary tree holds at most 2^(h+1)−1 nodes. Every tree therefore has $h \ge \log_2(n+1)-1$, giving Ω(log n) as n grows. Together, E[H_n]=Θ(log n).

Tail bounds and the scope of balancing guarantees

For c>3, nonnegative Z_n and Markov give

$$\Pr(H_n \ge c \log_2 n)=\Pr(Z_n \ge n^c)\le E[Z_n]/n^c \le b_n/n^c = O(n^{3-c}).$$

c=4 gives an O(1/n) tail; choose c≥k+3 for failure probability O(n^(−k)). This is a high-probability bound under random permutations, not a fixed height≤4log n for every ordinary-BST input.

Input 1..n still deterministically creates a height n−1 chain. Deletion followed by insertion need not preserve the random-construction distribution, so an insertion-only theorem does not cover arbitrary dynamic sequences. Worst-case O(log n) per operation requires an explicitly balanced invariant, such as the next chapter's red-black tree; vaguely random data is no proof.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 12.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study